Name: Hasini

Role: AI/Ml Intern

Day 5 - Task 1: Improved Body Landmark Detection

What Was Already There

The existing pose detection code used MediaPipe and OpenCV to detect 33 body landmarks in real time through the webcam. It drew a skeleton on the body and labeled 8 key points including shoulders, elbows, hips and knees on the screen.

What I Added

I added a real time posture analysis feature to the existing code. This feature automatically detects whether the person is standing in a good posture or bad posture and displays the result on the screen.

How the Posture Detection Logic Works

The system checks three conditions simultaneously using the detected landmark coordinates:

- **Shoulder Level Check:** Calculates the vertical difference between left shoulder and right shoulder. If the difference is more than 20 pixels the shoulders are considered uneven
- **Hip Level Check:** Calculates the vertical difference between left hip and right hip. If the difference is more than 20 pixels the hips are considered uneven
- **Body Alignment Check:** Calculates the horizontal difference between shoulder midpoint and hip midpoint. If the difference is more than 40 pixels the body is considered misaligned

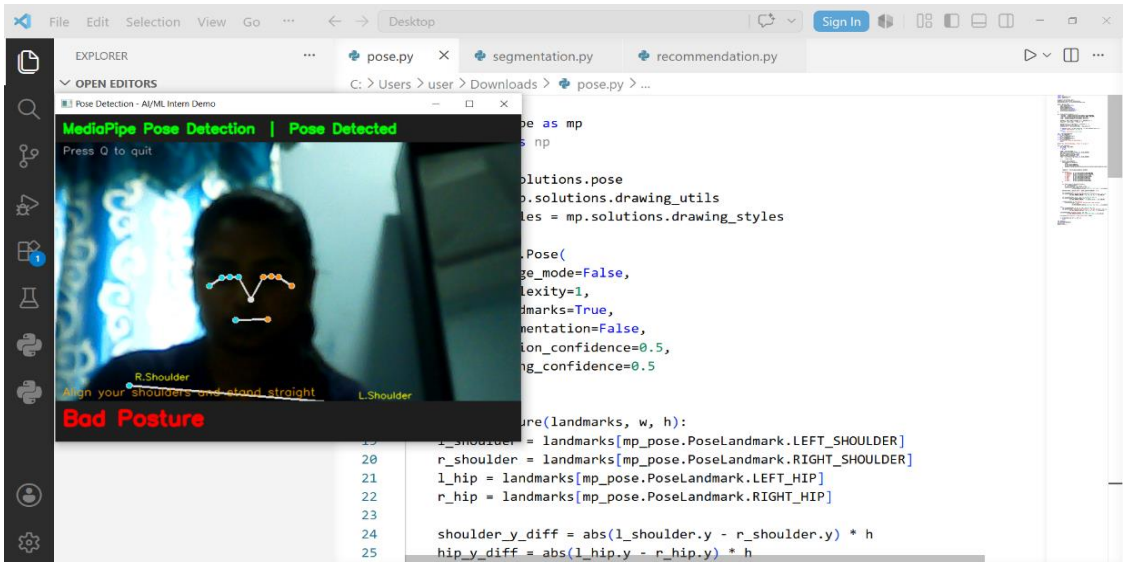
Formula used:

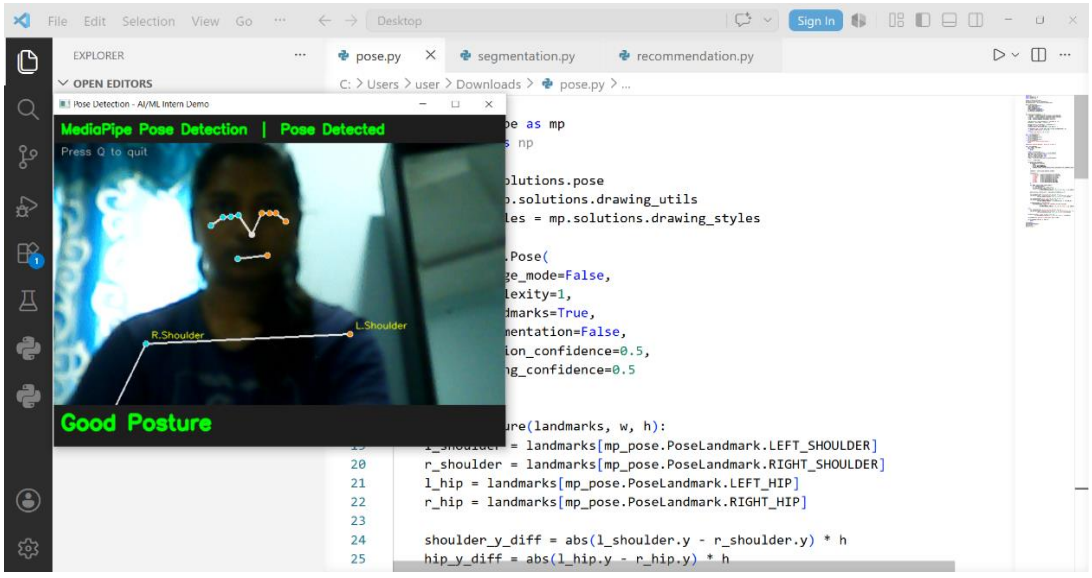
shoulder_y_diff = |left_shoulder.y - right_shoulder.y| × height
hip_y_diff = |left_hip.y - right_hip.y| × height
alignment_diff = |shoulder_mid_x - hip_mid_x| × width

Why This Feature is Useful

Posture detection is directly useful in virtual try-on systems because clothing fits and looks different depending on how a person is standing. If the person is leaning or slouching the garment overlay will not align correctly with the body. By ensuring the person is standing straight before taking measurements or applying virtual clothing the accuracy of the entire system improves significantly.

Output:


Pose Detection - AI/ML Intern Demo
MediaPipe Pose Detection | Pose Detected
Press Q to quit
R.Shoulder
L.Shoulder
Bad Posture
pose.py
segmentation.py
recommendation.py
OPEN EDITORS
r_shoulder = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER]
l_hip = landmarks[mp_pose.PoseLandmark.LEFT_HIP]
r_hip = landmarks[mp_pose.PoseLandmark.RIGHT_HIP]
shoulder_y_diff = abs(l_shoulder.y - r_shoulder.y) * h


Pose Detection - AI/ML Intern Demo
MediaPipe Pose Detection | Pose Detected
Press Q to quit
R.Shoulder
L.Shoulder
Good Posture
pose.py
segmentation.py
recommendation.py
OPEN EDITORS
r_shoulder = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER]
l_hip = landmarks[mp_pose.PoseLandmark.LEFT_HIP]
r_hip = landmarks[mp_pose.PoseLandmark.RIGHT_HIP]
shoulder_y_diff = abs(l_shoulder.y - r_shoulder.y) * h

Task 2 — Clothing Fitting Algorithms Research

A clothing fitting algorithm is a set of mathematical and AI based rules that determine which size and style of clothing will best fit a person based on their body measurements. Instead of relying on standard size charts like S, M, L, XL which vary across brands, AI fitting algorithms use actual body measurements extracted from pose landmarks to predict the exact fit for each individual person.

How It Works

The algorithm takes body measurements as input and maps them to clothing dimensions to find the best match.

- **Shoulder Width** — distance between left and right shoulder landmarks
- **Chest Width** — estimated from shoulder and torso landmarks
- **Waist Width** — estimated from hip landmarks with a ratio factor
- **Hip Width** — distance between left and right hip landmarks
- **Arm Length** — distance from shoulder to elbow to wrist landmarks
- **Inseam Length** — distance from hip to knee to ankle landmarks
- **Height** — total distance from head to ankle landmarks

Types of Fitting Algorithms

1. Rule Based Fitting

The simplest approach where fixed size charts are used to map measurements to sizes.

```
if shoulder_width < 38cm:
    size = "S"
elif shoulder_width < 42cm:
    size = "M"
elif shoulder_width < 46cm:
    size = "L"
else:
    size = "XL"
```

- Easy to implement
- Not very accurate across different brands
- Does not account for body proportions

2. Euclidean Distance Based Fitting

Calculates the distance between the user's measurements and standard size measurements to find the closest match.

$$\text{Distance} = \sqrt{(\text{user\_shoulder} - \text{size\_shoulder})^2 + (\text{user\_waist} - \text{size\_waist})^2 + (\text{user\_hip} - \text{size\_hip})^2}$$

The size with the smallest distance is recommended. More accurate than rule based because it considers multiple measurements together.

3. Machine Learning Based Fitting

Uses a trained model to predict the best fit based on thousands of user measurements and their feedback on fit quality.

- Input: body measurements extracted from pose landmarks
- Output: recommended size and fit type
- Model learns patterns from data like which measurements matter most for each clothing category
- Shirt fitting depends more on shoulder and chest
- Trouser fitting depends more on waist and inseam
- Dress fitting depends on all measurements together

4. 3D Body Model Fitting

The most advanced approach where a 3D avatar is generated from body measurements and the clothing is virtually draped onto it using physics simulation.

$$\text{3D Avatar} = f(\text{height, shoulder, waist, hip, arm\_length})$$

$$\text{Cloth Drape} = \text{Physics\_Simulation}(\text{Avatar, Garment\_3D\_Model})$$

- Most accurate and realistic
- Used by companies like Zara, H&M and Amazon
- Requires depth camera or dual camera setup for best results

Task 3 — Virtual Try-On Dataset Testing

Dataset Used: HR-VITON (High Resolution Virtual Try-On Network)

Dataset Structure

The dataset is organized into 9 folders inside the test directory, each serving a specific purpose in the virtual try-on pipeline:

- **image** — Original person photos (2032 images)
- **cloth** — Flat lay clothing images (2032 images)
- **cloth-mask** — Binary masks of clothing items (2032 images)
- **openpose-img** — Pose detection output images (2032 images)
- **openpose-json** — Pose landmark coordinates in JSON format (2032 files)
- **image-parse-v3** — Body segmentation maps (2032 images)
- **agnostic-v3.2** — Person images with clothing removed (2032 images)
- **image-densepose** — Dense body surface maps (2032 images)
- **image-parse-agnostic** — Segmentation without clothing region (2032 images)

File Count

- Total files in dataset: **18,405**
- Images per category: **2,032**
- Output try-on result images: **25**

Corrupted Image Check

- Images checked: **452**
- Corrupted files: **0**
- Null or bad files: **0**
- Status: **CLEAN**

No corrupted or missing files were found in the dataset confirming the dataset is complete and ready for model training and testing.

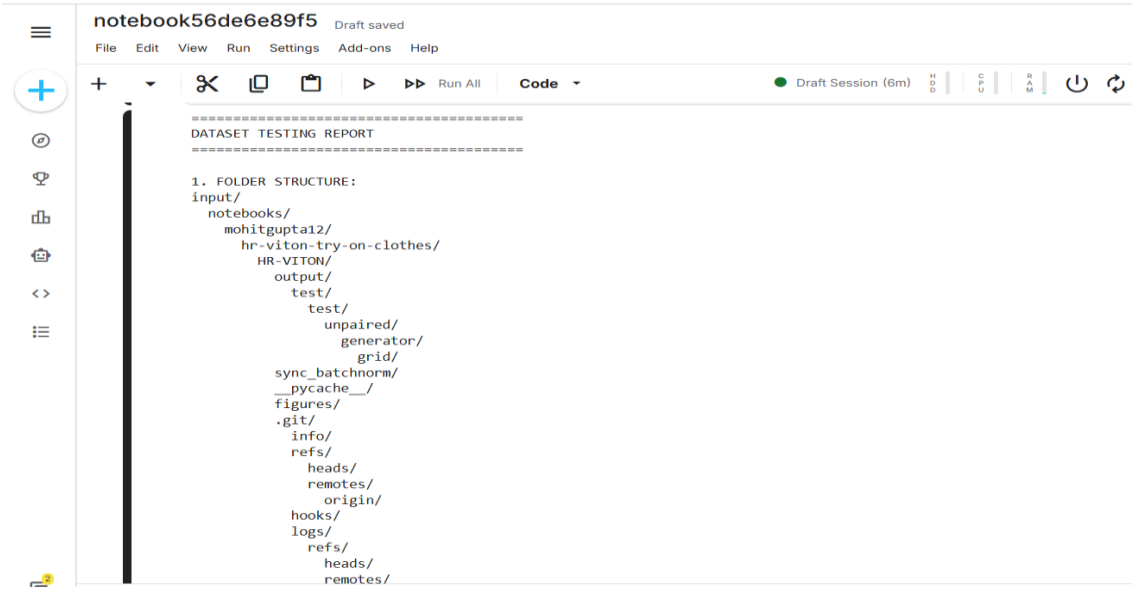
Image Size Analysis

- Minimum size: **716 x 1024 pixels**
- Maximum size: **5400 x 15442 pixels**
- Average size: **906 x 1171 pixels**

The dataset contains high resolution images which is important for virtual try-on because low resolution images produce blurry and unrealistic clothing overlays. The standard size used for training is 768 x 1024 pixels.

What I Understood from the Dataset

Each image in the dataset has a corresponding pair — a person image and a clothing image. The AI model takes both as input and produces a try-on result showing the person wearing that clothing. The openpose JSON files contain the exact body landmark coordinates which are used to align the clothing to the correct position on the body. The agnostic images are the person photos with the original clothing removed which is what the model uses as a base to place the new garment o



n.

notebook56de6e89f5 Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (6m)

```
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/__results___files: 1 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data: 1 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data/test/image-densepose: 2032 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data/test/agnostic-v3.2: 2032 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data/test/openpose_img: 2032 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data/test/image-parse-agnostic-v3.2: 2032 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data/test/openpose_json: 2032 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data/test/cloth: 2032 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data/test/cloth-mask: 2032 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data/test/image-parse-v3: 2032 files
  /kaggle/input/notebooks/mohitgupta12/hr-viton-try-on-clothes/data/test/image: 2032 files
  TOTAL FILES: 18405

3. CHECKING FOR CORRUPTED IMAGES:
  Images Checked : 452
  Corrupted Files: 0
  Null/Bad Files : 0

4. IMAGE SIZE ANALYSIS:
  Min Size : 716 x 1024
  Max Size : 5400 x 15442
  Avg Size : 906 x 1171

5. DATASET QUALITY:
  Total images verified : 452
  Corrupted             : 0
  Status: CLEAN — No corrupted files found

6. DISPLAYING SAMPLE IMAGES...
```

Sample Images from Dataset

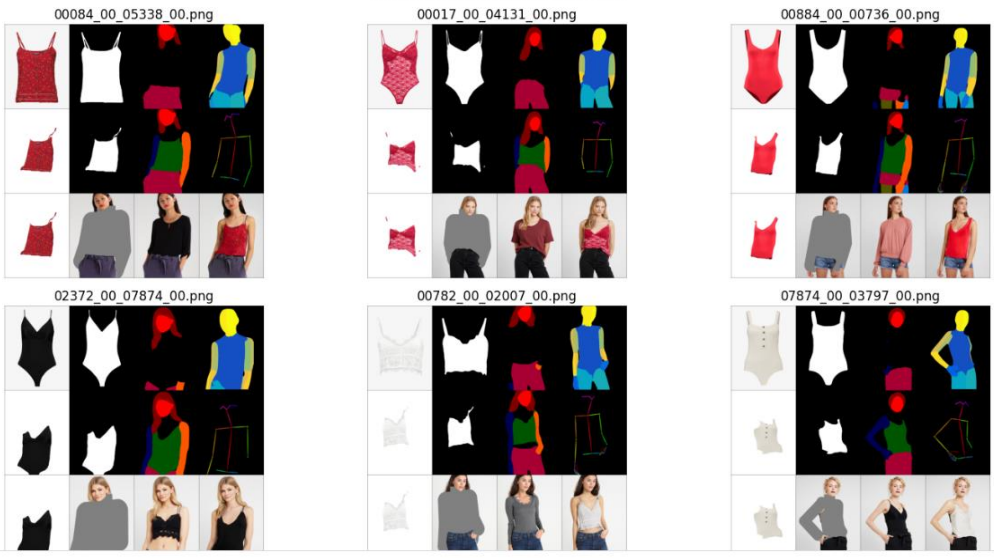
notebook56de6e89f5 Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (6m)

6. DISPLAYING SAMPLE IMAGES...

Sample Images from Dataset



Task 4 — Outfit Recommendation Logic

I created a rule based clothing recommendation system in Python that suggests outfits based on four user inputs — body type, occasion, weather and style preference.

GitHub Link: https://github.com/Hasinireddy2407/AI-ML-INTERNSHIP

The logic works in four layers:

- **Body Type Layer:** Determines the fit type — slim fit for slim body, regular fit for athletic body and relaxed fit for plus size body
- **Occasion Layer:** Determines the clothing category — formal wear, casual wear, party wear or sportswear based on where the person is going
- **Weather Layer:** Determines the fabric — cotton and linen for hot weather, wool and fleece for cold weather, quick dry fabric for rainy weather
- **Color Layer:** Suggests color palette — neutral, bright, pastel or dark colors based on preference

All four layers work together to give a final recommendation of outfit, fit, fabric and colors in one output.

Output:

```
==============================================
   AI Clothing Recommendation System
==============================================

User Preferences:
  Body Type: athletic
  Occasion: casual
  Weather: hot
  Preferred Style: western
  Preferred Color: neutral

Recommendations:
  Outfit     : T-Shirt + Jeans
  Fit        : regular fit
  Fabric     : Cotton / Linen
  Colors     : White, Black, Grey, Beige
  Avoid      : Avoid dark colors and heavy fabrics
==============================================
PS C:\Users\user\AppData\Local\Programs\Microsoft VS Code>
```

Task 5 — AI Avatar Generation Workflow

AI avatar generation is the process of creating a digital 3D representation of a human body from measurements or images. This digital body is called an avatar and it acts as a virtual mannequin on which clothes can be tried on, fitted and visualized without the person physically being present.

Working:

Step 1 — Input Collection

- A person stands in front of a camera in a standard pose called T-pose or A-pose
- Two photos are taken — front view and side view
- Or body measurements are entered manually like height, weight, shoulder width, waist and hip

Step 2 — Pose Estimation

- MediaPipe or OpenPose detects body landmarks from the input photos
- 33 key points are identified on the body
- These landmarks define the skeleton structure of the avatar

Step 3 — Body Shape Estimation

- A parametric body model called SMPL (Skinned Multi-Person Linear Model) is used
- SMPL represents the human body using two parameters:

Shape Parameter (β) — controls body shape: thin, athletic, plus size
Pose Parameter (θ) — controls body position and joint angles
Body = SMPL(β, θ)

- The model fits these parameters to match the detected landmarks

Step 4 — 3D Mesh Generation

- A 3D mesh is generated from the body parameters
- The mesh contains thousands of vertices that define the body surface
- This mesh is the actual 3D avatar

Step 5 — Texture Mapping

- Skin texture and appearance are mapped onto the 3D mesh
- The avatar now looks like a realistic human body

Step 6 — Clothing Overlay

- A 3D garment model is draped onto the avatar using physics simulation
- The cloth follows the body shape, gravity and movement

- Formula for cloth simulation:

$$F_{total} = F_{gravity} + F_{collision} + F_{internal}$$

Where:

- $F_{gravity}$ = downward pull on cloth
- $F_{collision}$ = cloth interaction with body surface
- $F_{internal}$ = cloth stiffness and elasticity

Tools and Technologies Used

- **SMPL Model** — parametric human body model used in most research
- **PIFuHD** — AI model that generates 3D avatar from a single photo
- **DensePose** — maps 2D image pixels to 3D body surface
- **Three.js / Blender** — used to render and display the 3D avatar
- **MediaPipe** — provides pose landmarks as input to the avatar system

Connection to Our Project

In our internship project the avatar generation workflow connects directly to what we have built so far:

- Pose detection (pose.py) → provides body landmarks
- Segmentation (segmentation.py) → removes background and isolates body
- Body measurements → fed into SMPL model to generate avatar shape
- Recommendation system → selects clothing to overlay on avatar
- Virtual try-on → final output showing person wearing recommended clothing